

UNIVERSIDAD NACIONAL AUTÓNOMA DE
MÉXICO

FACULTAD DE INGENIERÍA

Desarrollo de una FFT de 1024 Puntos Mediante Mariposas

Curso: Procesamiento Digital de Señales

Profesor: Dr. Oscar Francisco Fuentes Casarrubias

Alumno: Chirino Sánchez Luis Fernando (**Matrícula:** 321006314)

Alumno: Dominguez Calihua Fernando (**Matrícula:** 321248981)

Alumno: Garay Hernández Jorge Efrén (**Matrícula:** 321042545)

7 de octubre de 2026

1. Introducción

La Transformada Discreta de Fourier (DFT) es la fórmula matemática fundamental que permite tomar una señal discreta en el tiempo y obtener qué frecuencias contiene. Por otro lado, la Transformada Rápida de Fourier (FFT) no es una transformación matemática distinta a la DFT, sino una familia de algoritmos diseñados para calcular exactamente el mismo resultado numérico, pero de una forma mucho más rápida y con menos operaciones.

La razón principal para usar la FFT es la gran reducción en el costo de cálculo. Si calculamos la DFT directa de N puntos usando su sumatoria, se deben realizar multiplicaciones y sumas complejas para cada dato, lo que resulta en un costo de orden cuadrático, $\mathcal{O}(N^2)$. Cuando la cantidad de muestras N crece, este cálculo se vuelve demasiado lento para procesar señales en tiempo real.

Para resolver esto, la FFT divide el problema en partes más pequeñas. El algoritmo aprovecha que los factores de giro que multiplican a la señal se repiten y tienen simetrías en el plano complejo. Gracias a esto, no hace falta calcular multiplicaciones repetidas, logrando reducir la complejidad computacional a un orden de $\mathcal{O}(N \log_2 N)$.

2. Planteamiento del problema

La Transformada Discreta de Fourier para una secuencia discreta de N muestras, $x[n]$, se define formalmente mediante la siguiente ecuación:

$$X[k] = \sum_{n=0}^{N-1} x[n] W_N^{kn}, \quad \text{para } k = 0, 1, \dots, N-1 \quad (1)$$

donde el término $W_N = e^{-j\frac{2\pi}{N}}$ representa el factor de giro. Para este proyecto, el tamaño de la señal asignada es de $N = 1024$ puntos.

Para que el algoritmo clásico de FFT en base 2 (Radix-2) funcione de manera directa, la cantidad de muestras debe ser exactamente una potencia de 2. Al revisar nuestro caso, comprobamos que se cumple esta condición:

$$1024 = 2^{10} \quad (2)$$

Al aplicar $\log_2(1024) = 10$, sabemos que la arquitectura de nuestra FFT se organizará en exactamente 10 etapas sucesivas de cálculo. A lo largo de estas 10 etapas, las muestras se irán combinando mediante operaciones básicas de cruce (mariposas) hasta entregar los 1024 datos en frecuencia de manera eficiente.

3. Desarrollo por mariposas

El núcleo de cálculo de la FFT se basa en una estructura simple llamada mariposa.

3.1. Deducción matemática (Decimación en el Tiempo)

Para entender de dónde sale la mariposa, separamos la sumatoria original de la DFT en dos partes de tamaño $N/2$: las muestras con índice par ($n = 2m$) y las muestras con índice impar ($n = 2m + 1$):

$$\begin{aligned} X[k] &= \sum_{m=0}^{\frac{N}{2}-1} x[2m] W_N^{k(2m)} + \sum_{m=0}^{\frac{N}{2}-1} x[2m+1] W_N^{k(2m+1)} \\ &= \sum_{m=0}^{\frac{N}{2}-1} x[2m] W_{N/2}^{mk} + W_N^k \sum_{m=0}^{\frac{N}{2}-1} x[2m+1] W_{N/2}^{mk} \end{aligned} \quad (3)$$

Cada una de estas dos sumatorias es en realidad una DFT de tamaño $N/2$. Si llamamos $E[k]$ al resultado de los pares y $O[k]$ al de los impares, la expresión queda como:

$$X[k] = E[k] + W_N^k O[k] \quad (4)$$

3.2. Operación básica de mariposa

Dado que los resultados $E[k]$ y $O[k]$ se repiten a partir de la mitad del espectro, y sabiendo que el factor de giro cumple la simetría $W_N^{k+N/2} = -W_N^k$, la segunda mitad de frecuencias se calcula usando los mismos valores:

$$X_1 = A + B \cdot W_N^k \quad (5)$$

$$X_2 = A - B \cdot W_N^k \quad (6)$$

donde A es el dato par y B es el dato impar. Esta operación toma dos datos de entrada, multiplica el dato inferior por el factor de giro y entrega dos salidas mediante una suma y una resta. Por la forma cruzada de sus líneas, recibe el nombre de mariposa. Su gran ventaja es que con una sola multiplicación compleja se calculan dos puntos del espectro.

3.3. Factores de giro

El factor de giro se define matemáticamente como:

$$W_N^k = e^{-j\frac{2\pi k}{N}} = \cos\left(\frac{2\pi k}{N}\right) - j \sin\left(\frac{2\pi k}{N}\right) \quad (7)$$

Estos factores representan giros sobre el círculo unitario complejo. En la práctica, para que el algoritmo sea rápido en una computadora o procesador, estos números no se calculan a cada momento, sino que se dejan guardados con anticipación en una tabla en memoria.

3.4. Flujo de datos

Los datos avanzan de izquierda a derecha a lo largo de las 10 etapas. Cuando una mariposa termina su cálculo, guarda sus dos resultados exactamente en los mismos

espacios de memoria donde estaban sus dos datos de entrada. A esto se le conoce como cálculo en el mismo lugar (*in-place*), lo cual permite ahorrar memoria RAM porque no se requiere crear arreglos adicionales.

3.5. Implementación algorítmica de la mariposa y las etapas

Para llevar a la práctica la mariposa y el flujo entre etapas, el algoritmo se desglosa en código de la siguiente manera:

Unidad elemental de mariposa: La función recibe las dos muestras complejas (A y B) y el factor de giro actual W . Realiza únicamente una multiplicación compleja y genera las salidas sumando y restando:

```
1 def
2
3
4     return
```

Cálculo de etapas y factor de giro base: En el ciclo principal se recorren las 10 etapas ($m = 1, \dots, 10$). En cada nivel, el tamaño de bloque se duplica ($2, 4, 8, \dots$) y la distancia entre los datos a combinar es exactamente la mitad de ese bloque (2^{m-1}). El factor de giro base se obtiene dividiendo el círculo unitario entre el tamaño de bloque:

```
1 for     in range
2
3
4
5     # Factor de giro base para el bloque actual
6
```

Recorrido de grupos y ejecución in-place: Para cada grupo dentro de la etapa, el acumulador de giro arranca en $W = 1 + 0j$ (e^0). Se emparejan los índices superior e inferior separados por `mitad_bloque`, se ejecuta la mariposa y el resultado sobrescribe directamente las posiciones de entrada en memoria:

3.6. Implementación de la mariposa y las etapas en código

Para comprender cómo se traducen estos conceptos a un programa real, el algoritmo se desglosa en los siguientes fragmentos:

Unidad elemental de mariposa: Toma las entradas A y B , multiplica B por el factor de giro y genera las dos salidas sumando y restando:

```
def mariposa_2x2(a, b, factor_giro):
    salida_1 = a + factor_giro * b
    salida_2 = a - factor_giro * b
    return salida_1, salida_2
```

Avanzar por etapas y calcular el factor base: En cada etapa $m \in [1, 10]$, el tamaño de bloque se duplica (2^m) y la distancia entre muestras a combinar es `mitad_bloque` (2^{m-1}). La rotación base por mariposa se define con la exponencial compleja:

```
for etapa in range(1, etapas + 1):
    tamano_bloque = 2 ** etapa
    mitad_bloque = tamano_bloque // 2

    # Factor de giro base para el bloque actual
    W_N = np.exp(-1j * 2 * np.pi / tamano_bloque)
```

Recorrido de grupos y cálculo in-place: Se recorren los bloques de la señal. Al inicio de cada grupo el giro arranca en $W = 1 + 0j$, se combinan los datos correspondientes y los resultados sobrescriben el arreglo:

```
for k in range(0, N, tamano_bloque):
    W = 1 + 0j

    for j in range(mitad_bloque):
        indice_superior = k + j
        indice_inferior = k + j + mitad_bloque

        X[indice_superior], X[indice_inferior] = mariposa_2x2(
            X[indice_superior],
            X[indice_inferior],
            W
        )

    # Actualizacion acumulativa del giro
    W = W * W_N
```

4. Estructura de las 10 etapas

Para procesar las 1024 muestras, el algoritmo se organiza en 10 niveles o etapas sucesivas:

4.1. Número de mariposas por etapa

En cada etapa intervienen todas las muestras de la señal. Como cada mariposa procesa dos datos a la vez, la cantidad de mariposas en cada una de las 10 etapas es constante:

$$\frac{N}{2} = \frac{1024}{2} = 512 \text{ mariposas por etapa} \quad (8)$$

Al sumar las 10 etapas, el algoritmo completo ejecuta un total de $512 \times 10 = 5,120$ mariposas.

4.2. Evolución de las distancias entre índices

La distancia o salto entre los dos datos que entran a una misma mariposa va cambiando en cada etapa m siguiendo potencias de dos (2^{m-1}):

- **Etapas 1:** La distancia es de $2^0 = 1$ posición (se combinan muestras que están juntas).
- **Etapas 2:** La distancia se duplica a $2^1 = 2$ posiciones.
- **Etapas 3:** La distancia es de $2^2 = 4$ posiciones.
- **Etapas intermedias (4 a 9):** Las distancias continúan duplicándose: 8, 16, 32, 64, 128 y 256 posiciones.
- **Etapas 10:** La distancia llega a $2^9 = 512$ posiciones, combinando datos de la primera mitad con datos de la segunda mitad del arreglo.

Tabla 1: Organización de las 10 etapas para la FFT de 1024 puntos.

Etapas (m)	Distancia (2^{m-1})	Grupos	Mariposas/Grupo	Exponentes de Giro (k)
1	1	512	1	$\{0\}$
2	2	256	2	$\{0, 256\}$
3	4	128	4	$\{0, 128, 256, 384\}$
4	8	64	8	$\{0, 64, \dots, 448\}$
5	16	32	16	múltiplos de 32
6	32	16	32	múltiplos de 16
7	64	8	64	múltiplos de 8
8	128	4	128	múltiplos de 4
9	256	2	256	múltiplos de 2
10	512	1	512	$\{0, 1, 2, \dots, 511\}$

5. Ordenamiento de datos

5.1. Inversión de bits (Bit-Reversal)

Al separar repetidamente los datos en pares e impares, las muestras no pueden entrar a la primera etapa en su orden de tiempo normal. Para que el cálculo funcione y pueda sobrescribir la memoria de forma ordenada, la señal de entrada debe cambiar de posiciones antes de empezar.

Para saber en qué lugar colocar cada dato, el número de posición original (del 0 al 1023) se escribe en binario usando 10 bits y luego esos bits se colocan al revés (el primero pasa al final y el último al principio):

$$n = (b_9 b_8 b_7 b_6 b_5 b_4 b_3 b_2 b_1 b_0)_2 \longrightarrow n_{\text{rev}} = (b_0 b_1 b_2 b_3 b_4 b_5 b_6 b_7 b_8 b_9)_2 \quad (9)$$

Por ejemplo:

- La muestra 1 (00000000001_2) se mueve a la posición 512 (10000000000_2).
- La muestra 2 (00000000010_2) se mueve a la posición 256 (01000000000_2).
- La muestra 3 (00000000011_2) se mueve a la posición 768 (11000000000_2).

Al cambiar el orden de las muestras al inicio, los coeficientes de frecuencia a la salida de la etapa 10 terminan quedando ordenados de manera natural ($X[0], X[1], \dots, X[1023]$).

Implementación de la inversión de bits en código: La función calcula la cantidad de bits con $\log_2(N)$, convierte cada índice a cadena binaria, invierte dicha cadena mediante y reubica la muestra:

```
def ordenamiento_inversion_bits(x):
    N = len(x)
    num_bits = int(np.log2(N))
    x_reordenado = np.zeros(N, dtype=complex)
    for i in range(N):
        bit_string = '{:0{width}b}'.format(i, width=num_bits)
        indice_invertido = int(bit_string[::-1], 2)
        x_reordenado[indice_invertido] = x[i]
    return x_reordenado
```

5.2. Caso de decimación en frecuencia (DIF)

Como alternativa, si el algoritmo se diseña usando decimación en frecuencia, las muestras de entrada entran en su orden de tiempo normal ($0, 1, 2, \dots$). Sin embargo, las salidas después de la etapa 10 terminan cambiadas de lugar, por lo que el proceso de inversión de bits tiene que aplicarse al final para leer las frecuencias correctamente.

6. Diagrama o esquema

Para mostrar de forma clara el funcionamiento del algoritmo, se presentan dos esquemas: el flujo general a través de las 10 etapas del sistema y la operación interna de la mariposa elemental Radix-2 DIT.

7. Complejidad computacional

7.1. Costo de la DFT directa

Si calculamos la DFT de 1024 puntos evaluando directamente la sumatoria, se tienen que hacer 1024 multiplicaciones complejas y 1023 sumas complejas para cada uno de los 1024 puntos de frecuencia:

$$\text{Multiplicaciones complejas} = N^2 = (1024)^2 = 1,048,576 \quad (10)$$

$$\text{Sumas complejas} = N(N - 1) = 1024 \times 1023 = 1,047,552 \quad (11)$$

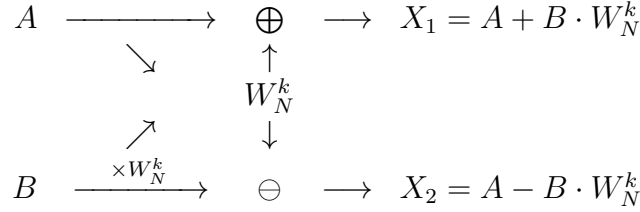
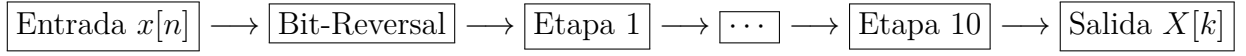


Figura 1: Diagrama de bloques del flujo en 10 etapas y estructura operativa de la mariposa Radix-2 DIT.

7.2. Costo de la FFT y ahorro de operaciones

En la FFT, cada una de las 512 mariposas realiza únicamente 1 multiplicación compleja y 2 sumas complejas por etapa. A lo largo de las 10 etapas, el costo total es:

$$\text{Multiplicaciones complejas} = \frac{N}{2} \log_2(N) = 512 \times 10 = 5,120 \quad (12)$$

$$\text{Sumas complejas} = N \log_2(N) = 1024 \times 10 = 10,240 \quad (13)$$

Tabla 2: Comparación de operaciones para una señal de 1024 puntos.

Operación	DFT Directa	FFT Radix-2	Ahorro Aprox.
Multiplicaciones Complejas	1,048,576	5,120	≈ 205 veces menos
Sumas Complejas	1,047,552	10,240	≈ 102 veces menos
Total Operaciones	2,096,128	15,360	≈ 136 veces menos

Al comparar ambas opciones, vemos que la FFT realiza casi 205 veces menos multiplicaciones complejas que la DFT directa. Este ahorro tan grande en operaciones matemáticas es lo que permite que el cálculo se pueda realizar en tiempo real en sistemas embebidos, computadoras y procesadores de señales.

8. Conclusiones

La realización de una FFT de 1024 puntos permite observar de manera clara cómo una Transformada Discreta de Fourier puede dividirse en operaciones más pequeñas mediante el uso de mariposas. Al cumplirse que $1024 = 2^{10}$, el algoritmo radix-2 requiere exactamente 10 etapas de procesamiento, en las cuales se combinan progresivamente las muestras hasta obtener los coeficientes espectrales.

El uso de los factores de giro permite realizar las operaciones de manera organizada, aprovechando las propiedades de periodicidad y simetría de la transformada. Además, el ordenamiento por inversión de bits es necesario en el método por decimación en tiempo, ya que permite que los datos ingresen en el orden adecuado para ir calculando y sobrescribiendo en la misma memoria.

En cuanto al costo computacional, la principal ventaja de la FFT consiste en reducir la complejidad de la DFT directa de $\mathcal{O}(N^2)$ a $\mathcal{O}(N \log_2 N)$. Para 1024 puntos, esto significa pasar de más de un millón de multiplicaciones a solo 5,120, lo que representa una reducción enorme en tiempo de cómputo.

Finalmente, la estructura estudiada muestra que la FFT es una herramienta totalmente viable para implementarse tanto en software como en hardware (por ejemplo en microcontroladores, DSPs o FPGAs). Su organización ordenada por etapas y mariposas permite aprovechar eficientemente los recursos de procesamiento, por lo que es la técnica estándar para aplicaciones de procesamiento digital de señales y análisis espectral.